

RACTF 2020 Crypto - Really Simple Algorithm

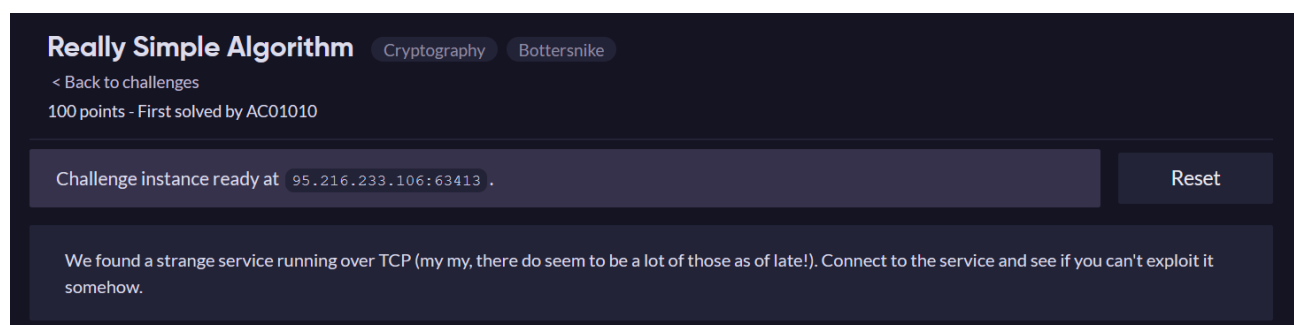
deuterium — 2020-06-09

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/ractf/crypto/Really'Simple'Algorithm/2020-06-09-RACTF-2020-Really-Simple-Algorithm.html>

Abstract

CTF writeup: RACTF 2020's Really Simple Algorithm ships RSA with p and q included. Compute the private exponent and decrypt; the factoring department has the day off.

Really Simple Algorithm



This is a typical RSA challenge. On netcatting the server, we get

```
p: 692978822802216497910263439691526372004023822242590405470708610553024726
902107848499618752687457377038930308135719449550272132309308464475108356502
3419193
q: 818545152008458581431308715472370387421587511434432009344550954468777476
263587958368173933311893826927568771762410426498881666412168638407642679708
2583709
e: 65537
ct: 32339597696112020672456048174497066278381984032062108835179880038510430
273215133772725151255048701614260554686353233886359332147981662885894807440
689371006779234552225971993752542529236343547917953433930922604820751362507
992551012513309785904323711107093361652451623376766542104555644651054024466
081454822549
```

Now, following wikipedia article, one could understand, in order to solve the challenge, one need to follow the following steps :-

- calculate $n = p \cdot q$
- calculate phi of n which is euler's totient which denotes the number of positive integers, which are relatively prime ($\text{GCD}=1$) with n.
In our case, it is calculated as $\phi = (p - 1) * (q - 1)$
- calculate the decryption key d which is modular inverse of encryption key e over phi
It is calculated using extended gcd. For our case, we can find the implementation in gmpy2
- $d = \text{gmpy2.invert}(e, \phi)$

- Once you have d, one can easily calculate plaintext as ciphertext raised to the power d modulo n
- `pt = pow(ct, d, n)`
- converting pt from integer representation to a string or byte-string representation
- `plaintext = bytes.fromhex(hex(pt)[2:])` or equivalently `plaintext = int.to_bytes(pt, (pt.bit_length()+7)//8, 'big')`
- print the plaintext, `print(plaintext.decode())`

Now, this process can be automated too, but since time is not concerned with this task as with the next Really Speedy Algorithm, I like to do it anyways

```
from pwn import remote
from gmpy2 import invert

HOST, PORT = "95.216.233.106", 20391

REM = remote(HOST, PORT)

p = int(REM.recvline().decode().strip()[3:])
q = int(REM.recvline().decode().strip()[3:])
e = int(REM.recvline().decode().strip()[3:])
ct = int(REM.recvline().decode().strip()[4:])

PHI = (p - 1) * (q - 1)
D = invert(e, PHI)
MESSAGE = pow(ct, D, p * q)
print(bytes.fromhex(hex(MESSAGE)[2:]).decode())
```

And we get our flag

ractf-JustLikeInTheSimulations"

| Interactive controls are omitted from this PDF. See the canonical HTML version.